Numerical Analysis Midterm Project

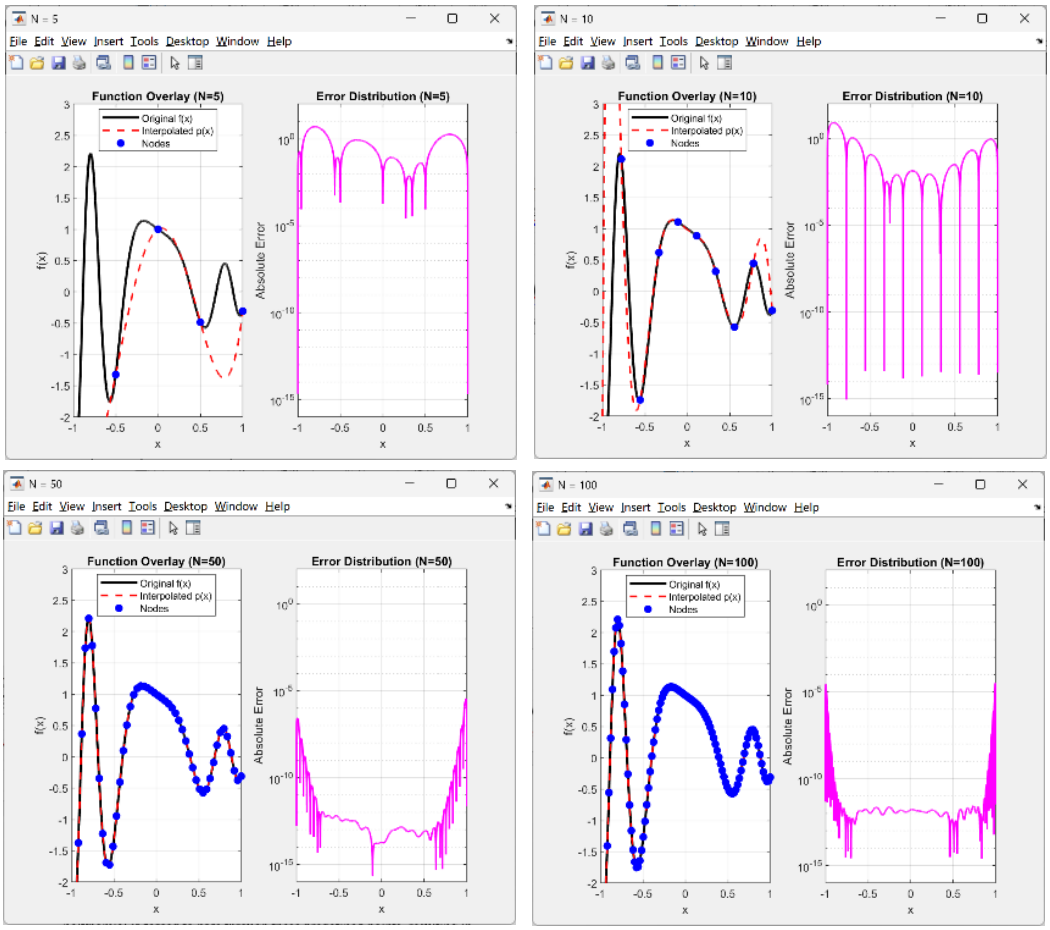
B12502027 李承軒

（本報告內容皆由 AI 協助翻譯，並由本人進行刪減和修改，

而原始中文版本將會附於最後面之附錄中）

1.

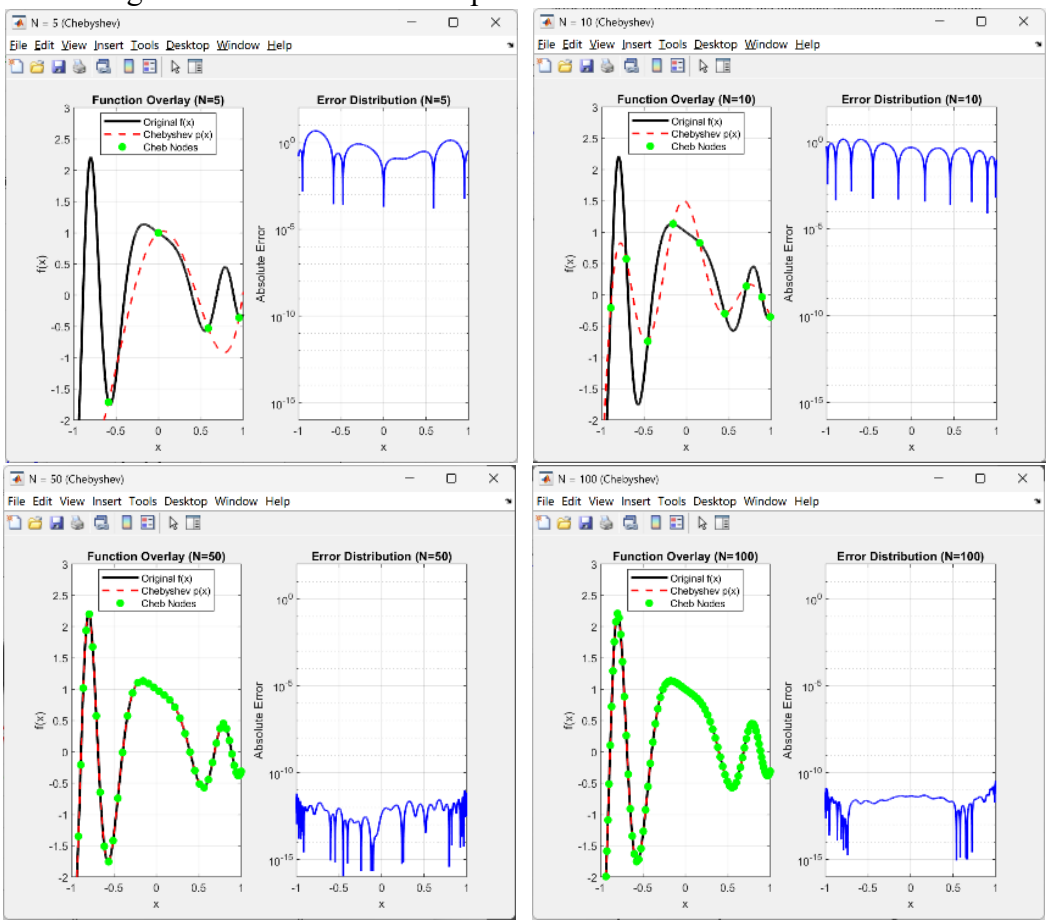
(a) .Following are the error distribution plots:



The numerical experiments conducted for interpolation point counts of $N = 5, 10, 50, 100$ provide a detailed perspective on how the distribution of points affects accuracy. In all four cases, it is evident that the absolute error approaches zero precisely at the interpolation nodes, as the polynomial is strictly constrained to pass through these coordinates. These instances manifest as sharp dips in the error distribution plots; however, due to the finite density of the computational grid, the error values at these nodes do not vanish entirely but remain at the level of machine precision. Excluding these nodal points, a distinct trend of "low error in the center, high error at the boundaries" becomes increasingly prominent as N increases. This indicates that while

increasing the number of interpolation points effectively stabilizes the error in the central region ($x \approx 0$), the requirement to force-fit evenly spaced nodes near the edges ($x \approx \pm 1$) causes obvious oscillations between those nodes, which leads to less reliable predictions in the boundary regions. In summary, simply increasing the number of interpolation points does not guarantee global uniform convergence, even if the goal is to enhance overall precision. Furthermore, upon a deeper investigation into Runge's Phenomenon, I speculate that the characteristics observed in these specific plots do not strictly constitute the phenomenon itself. Runge's phenomenon typically refers to a scenario where increasing the number of equidistant points causes errors at the boundaries to skyrocket to magnitudes such as 10^2 or 10^3. Observation of the $N = 100$ plot reveals that although the boundary errors are higher than those in the center—mirroring the distribution pattern associated with Runge's findings—the maximum error remains below a certain threshold and does not exhibit an uncontrolled exponential surge. Therefore, for this specific equation, while the evenly spaced sampling method results in a non-uniform error distribution, it does not trigger the divergent instability characteristic of Runge's phenomenon.

(b) . Following are the error distribution plots:



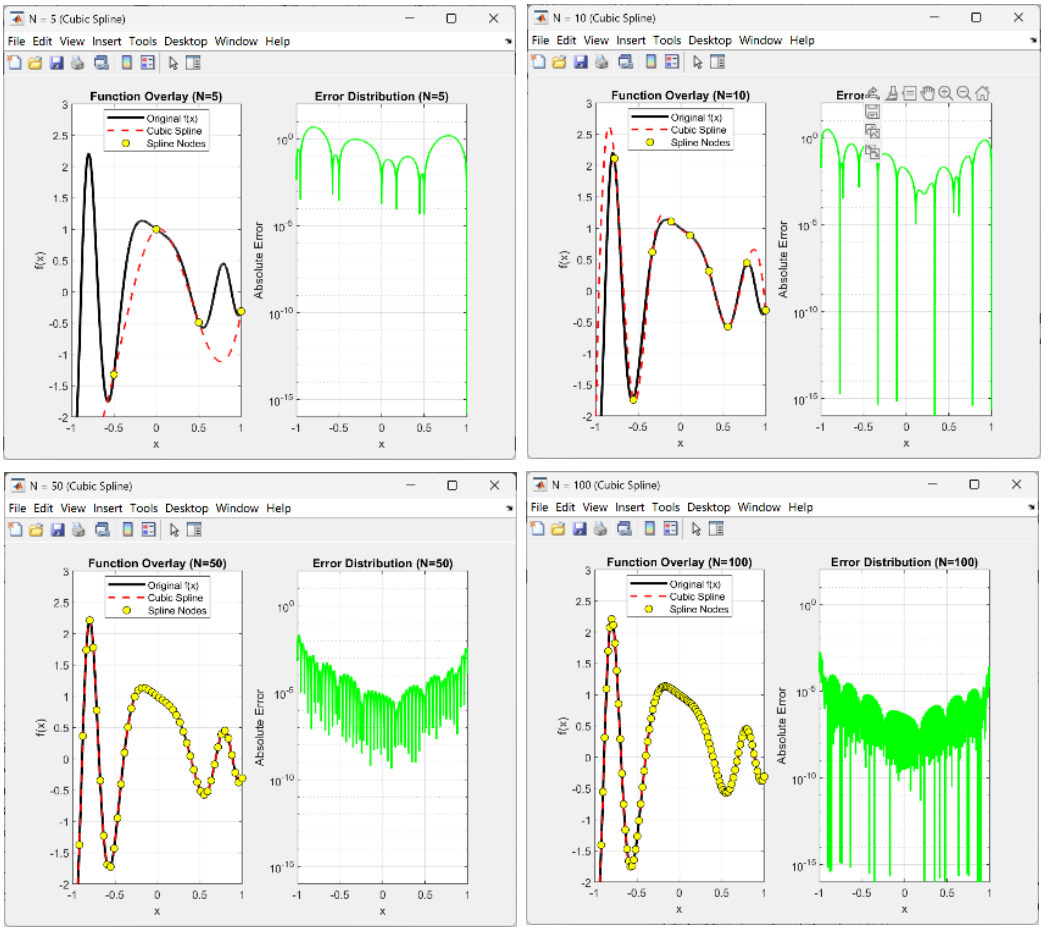
Similar to part (a), the error distributions were analyzed for $N = 5, 10, 50, 100$. In contrast to the uniformly distributed nodes used previously, Chebyshev nodes are defined as the projections of equidistant points from a semicircle onto a straight line. This geometric property creates a distribution that is sparse in the center and dense at the boundaries.

Based on the error distribution plots, it is evident that this specific node arrangement effectively suppresses oscillations by increasing the sampling density in the edge regions. This significantly mitigates the issue of error growth at the boundaries. Taking $N = 100$ as an example: while the error near the boundaries approaches 1 when using evenly spaced nodes, the error is controlled within the magnitude of 10^{-11} when utilizing Chebyshev nodes.

In summary, the use of Chebyshev nodes leads to a state of global convergence as N increases; the error diminishes uniformly across both the central and boundary regions, effectively suppressing the issue of non-uniform error distribution. This results in a much more stable and accurate approximation, proving that the choice of node distribution is critical to the stability of high-degree polynomial interpolation.

Furthermore, if the original function is susceptible to Runge's Phenomenon at the boundaries, this sampling method can avoid it quite effectively. The dense distribution of nodes near the edges significantly alleviates the error spikes characteristic of Runge's Phenomenon, yielding much more robust results than equidistant sampling.

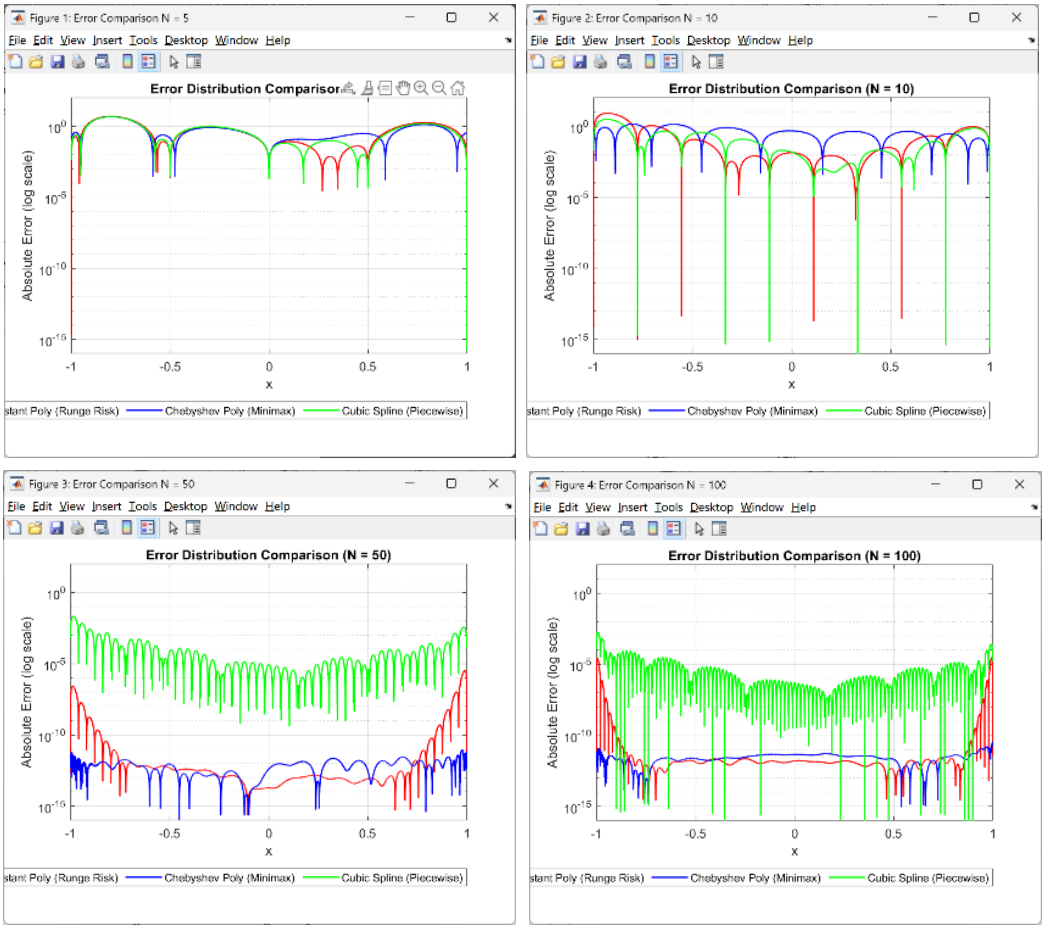
(c) . Following are the error distribution plots:



In contrast to the high-degree polynomial interpolation in part (a), which utilizes a single global curve, Cubic Spline Interpolation constructs a cubic polynomial between each pair of adjacent nodes. This approach ensures the continuity of both the first and second derivatives at the nodal points.

As observed from the four error distribution plots, although a slight "low in the center, high at the edges" trend persists as N increases, Cubic Spline Interpolation effectively mitigates the issue of excessive errors in the boundary regions compared to the results in part (a). This method successfully constrains the global error distribution within a controlled range. Furthermore, as N increases, the global error exhibits a uniform decline, demonstrating that when employing equidistant sampling, piecewise low-degree polynomials offer significantly higher stability than a single high-degree polynomial. Similar to the Chebyshev node method, piecewise low-degree polynomials are also effective at reducing the massive oscillations and errors associated with Runge's Phenomenon near the edges.

Error Discussion Across Three Methods:



Finally, an overlay of the error distributions for all three methods is presented for cross-comparison. It is evident that at low values of N, the errors across the three methods are comparable, with magnitudes hovering around 10^1. However, as N increases beyond 50, the two single high-degree polynomial methods yield significantly lower errors in the central region compared to the piecewise low-degree method, with a difference approaching 10^{-9} orders of magnitude.

Nevertheless, regarding the uniformity of error distribution, both Chebyshev nodes and Cubic Spline Interpolation significantly outperform equidistant nodes. As N approaches 100, the boundary errors of the equidistant method even become comparable to the overall error level of the Cubic Spline Interpolation. In conclusion, while single high-degree polynomials offer superior precision in the central segment as N increases, Chebyshev nodes and Cubic Spline Interpolation provide much better global error consistency once N is large enough to keep all errors within a controllable range.

2.

(a) . $g(x;a) = e^{-ax^2}$ and $f(x;a) = \sum_{m=-\infty}^{\infty} g(x - 2\pi m; a)$

For the continuous Fourier transform of $g(x;a)$, $G(s;a)$:

$$G(s;a) = \int_{-\infty}^{\infty} g(x;a)e^{-isx}dx = \int_{-\infty}^{\infty} e^{-ax^2}e^{-isx}dx$$

$$-ax^2 - isx = -a\left(x^2 + \frac{is}{a}x\right) = -a\left(x + \frac{is}{2a}\right)^2 - \frac{s^2}{4a}$$

Substitute into the integral → $G(s;a) = e^{-\frac{s^2}{4a}} \int_{-\infty}^{\infty} e^{-a(x+\frac{is}{2a})^2}dx$

Knowing that $\int_{-\infty}^{\infty} e^{-au^2}du = \sqrt{\frac{\pi}{a}}$ → $G(s;a) = \sqrt{\frac{\pi}{a}}e^{-\frac{s^2}{4a}}$

Analytical expression for the Fourier series coefficients of $f(x;a)$, $\hat{f}_j(a)$:

$f(x;a) = \sum_{j=-\infty}^{\infty} \hat{f}_j(a)e^{ijx}$

$$\hat{f}_j(a) = \frac{1}{2\pi}\int_{-\pi}^{\pi} f(x;a)e^{-ijx}dx = \frac{1}{2\pi}\sum_{m=-\infty}^{\infty}\int_{-\pi}^{\pi} g(x - 2\pi m; a)e^{-ijx}dx$$

Let $u = x - 2\pi m, du = dx$

$$\rightarrow \hat{f}_j(a) = \frac{1}{2\pi}\sum_{m=-\infty}^{\infty}\int_{-\pi-2\pi m}^{\pi-2\pi m} g(u;a)e^{-iju}e^{-ij2\pi m}du$$

$$= \frac{1}{2\pi}\sum_{m=-\infty}^{\infty}\int_{-\pi-2\pi m}^{\pi-2\pi m} g(u;a)e^{-iju}du \quad (j, m \in \mathbb{Z}, e^{-ij2\pi m} = 1)$$

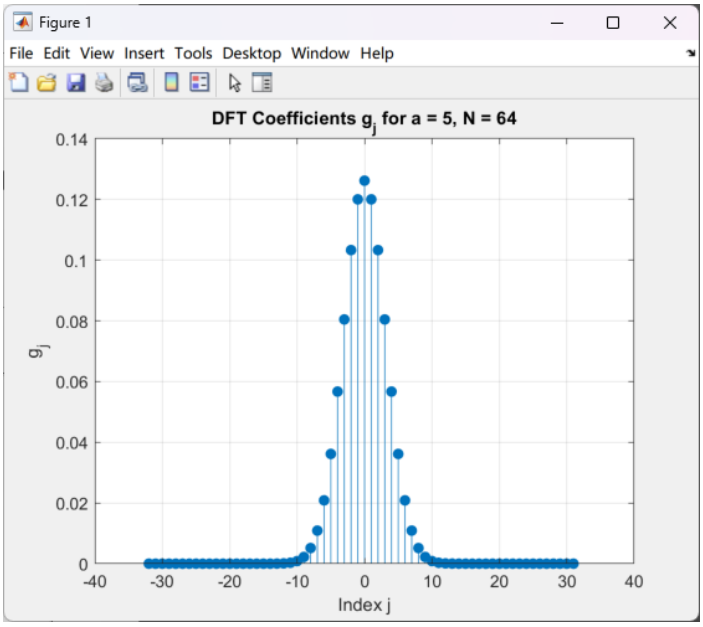
$$= \frac{1}{2\pi}\int_{-\infty}^{\infty} g(u;a)e^{-iju}du$$

Observing the two integral $\begin{cases} G(s;a) = \int_{-\infty}^{\infty} g(x;a)e^{-isx}dx \\ \hat{f}_j(a) = \frac{1}{2\pi}\int_{-\infty}^{\infty} g(u;a)e^{-iju}du \end{cases}$ it's easy to find out that $\hat{f}_j(a)$ are the value of $G(s;a)$ at the frequency point $s = j$, scaled by the factor of 2π

$$\rightarrow \hat{f}_j(a) = \frac{1}{2\pi}G(j;a) = \frac{1}{2\pi}\sqrt{\frac{\pi}{a}}e^{-\frac{j^2}{4a}}$$

(b) . Following are the DFT coefficients of $g(x;a)$:

Take a = 5, N = 64 for example

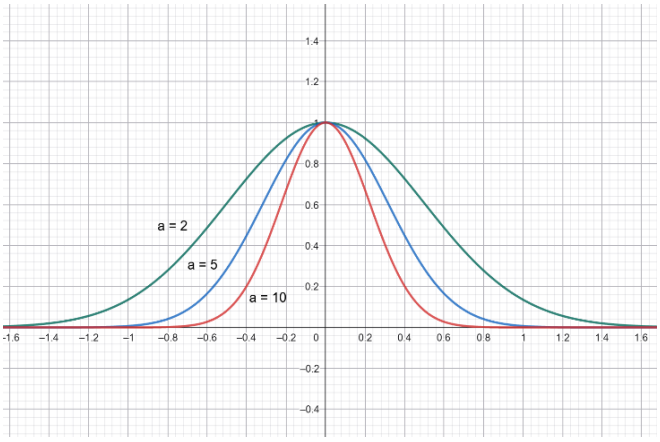


Verification:

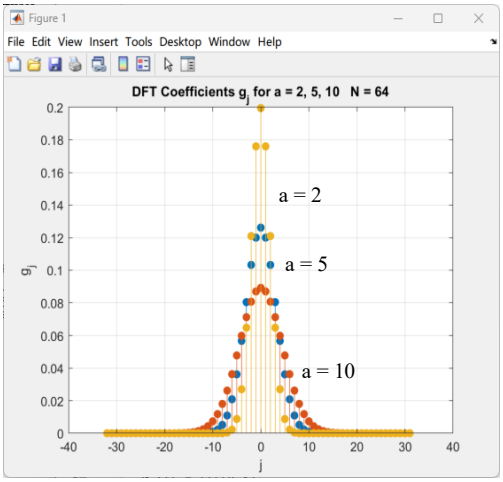
Based on the analytical expression derived in part (a), by substituting the parameters a = 5 and N = 64 into the formula, we can calculate the coefficient value at j = 0 equals to $\hat{f}_j(5) = \frac{1}{2\pi}\sqrt{\frac{\pi}{5}}e^{-\frac{0^2}{4\times5}} \approx 0.126$. The resulting value is found to be closely consistent with the peak height observed at j = 0 in the output plot.

The following section presents the output and comparison of the DFT coefficient plots for a = 2, 5, and 10 while keeping N = 64 as constant:

$g(x;a)$ plots



DFT coefficients $\hat{g}_j(a;N)$



It can be observed that as the parameter a increases, the time-domain function $g(x;a)$ becomes increasingly localized and sharper. Consequently, the

distribution of the DFT coefficients $\hat{g}_j(a;N)$ (representing the frequency domain) becomes broader and flatter. This phenomenon is a direct manifestation of the scaling property of Fourier Transforms

(c) . To isolate and analyze the three types of potential errors—truncation of the Fourier coefficients, sampling/aliasing, and non-periodicity/leakage—we must compare the following three values:

- $\frac{1}{2\pi}|G(s;a)|$: The continuous Fourier Transform (the ideal theoretical value without any numerical errors).
- $\left|\hat{f}_j(a)\right|$: The Fourier series coefficients.
- $\left|\hat{g}_j(a;N)\right|$: The DFT coefficients obtained via numerical computation.

Truncation Error:

Cause: The true continuous transform $G(s;a)$ extends infinitely across the frequency domain. However, when calculating the Fourier series, we only retain a finite range of frequencies $j \in (-\frac{N}{2}, \frac{N}{2}-1)$. The error generated by discarding the high-frequency tail is the Truncation Error, which causes the difference between the ideal value $\frac{1}{2\pi}|G(s;a)|$ and the series coefficients $\frac{1}{2\pi}|G(s;a)|$.

Isolation of Error: By fixing N at a sufficiently large value and gradually increasing a from a small value, we can observe the effect. When a is very small, the function decays slowly in the frequency domain, leading to a significant difference between $\left|\hat{f}_j(a)\right|$ and $\frac{1}{2\pi}|G(s;a)|$ if high-frequency regions are neglected. As a increases, the decay rate in the high-frequency region increases, and the difference between $\left|\hat{f}_j(a)\right|$ and $\frac{1}{2\pi}|G(s;a)|$ diminishes noticeably. This demonstrates that the parameter a controls the Truncation Error.

Sampling/Aliasing Error:

Cause: The calculation of $\left|\hat{f}_j(a)\right|$ still assumes a continuous (integral) form, whereas the calculation of $\left|\hat{g}_j(a;N)\right|$ only samples N discrete points within a fixed range. According to the Nyquist Theorem, if the signal

bandwidth exceeds the sampling range, high-frequency components will "fold back" and disguise themselves as lower frequencies, causing interference between different frequencies. The existence of this error can be found by comparing $\left|\hat{f}_j(a)\right|$ and $\left|\hat{g}_j(a;N)\right|$ at lower frequencies.

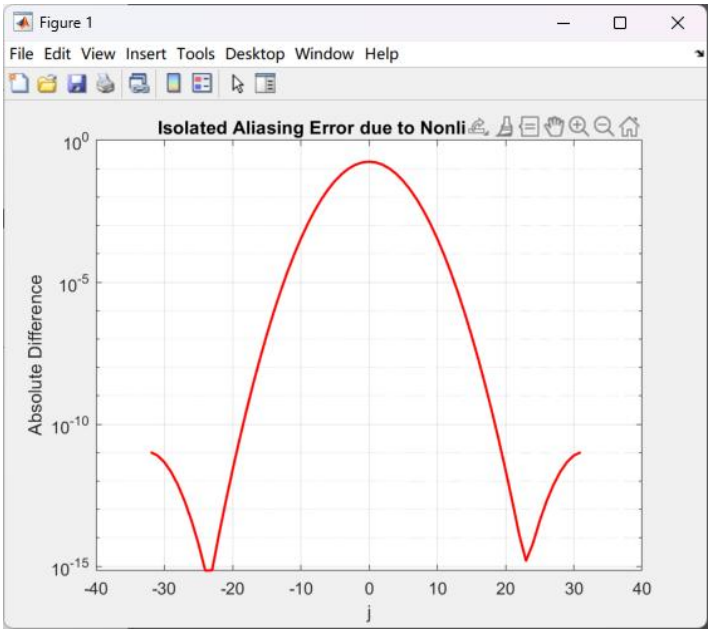
Isolation of Error: By fixing a large a (e.g., a = 10) and gradually increasing N, we can observe the trend. When N is small, the observation range is narrow. Since a=10 results in a wider spectrum, high-frequency components fold back and superimpose onto the low frequencies, causing $\left|\hat{g}_j(a;N)\right|$ at low frequencies to be significantly higher than the theoretical $\left|\hat{f}_j(a)\right|$. As N increases, the portion of the spectrum falling outside the range diminishes, and the aliasing error drops rapidly.

Non-periodicity/Leakage Error:

Cause: While the continuous function $G(s;a)$ is smooth and unbounded, the computation of the DFT coefficients $\left|\hat{g}_j(a;N)\right|$ is affected by the boundaries. If a discontinuity occurs when the signal is stitched together at the boundaries during periodic summation, a gap is created. According to Fourier analysis, any sharp discontinuity requires an infinite summation of high-frequency waves. This discontinuity subsequently disperses energy—which should originally be concentrated at specific frequencies—causing it to leak across the entire spectrum.

Isolation of Error: By fixing the value of N and gradually increasing a, we can isolate this effect. When a is small, the Gaussian function is relatively flat, potentially resulting in non-zero values at the boundaries that cause discontinuities. In this case, an overall upward shift in the error distribution can be observed. However, by increasing a, the Gaussian pulse becomes sharper, ensuring that it decays toward zero before reaching the boundaries. This effectively reduces the leakage error. Additionally, a window function can be applied to the signal to forcibly bring the values at both endpoints to zero. This ensures the periodicity of the signal within the observed domain, thereby effectively suppressing spectral leakage.

(d) . Following are the error due to the nonlinear product:



In this section, we first perform a direct FFT to obtain the DFT coefficients $\left|\tilde{h}_{j,1}(a;N)\right|$. We then recompute the coefficients $\left|\tilde{h}_{j,2}(a;N)\right|$ after applying the 3/2 De-aliasing Rule. By comparing the difference between these two, we can isolate the nonlinear aliasing error.

Direct Calculation ($\left|\tilde{h}_{j,1}(a;N)\right|$):

Assuming the maximum frequency component of $g(x;a)$ is k_{max}, the maximum frequency of its square $h(x;a) = g^2(x;a)$ becomes $k'_{max} = 2k_{max}$. If the original N sampling points are maintained, according to the Nyquist Theorem discussed in part (c), these high-frequency components exceeding the range will fold back into the low-frequency region, causing significant aliasing errors.

Calculation via the 3/2 De-aliasing Rule ($\left|\tilde{h}_{j,2}(a;N)\right|$):

The primary logic is to first perform a length N FFT. After transformation, the N spectral coefficients are placed into a larger array of length M = 3/2N, with zeros padded in the high-frequency regions (zero-padding). Performing an IFFT on these length M yields a finer sampling grid in the time domain. By conducting calculations on this refined grid, the extra high-frequency components have enough range to avoid folding back. Subsequently, a length M FFT is performed, followed by truncating the extra portions to retain only the coefficients corresponding to the original length N. Through this method, high-frequency components will not alias into the low-

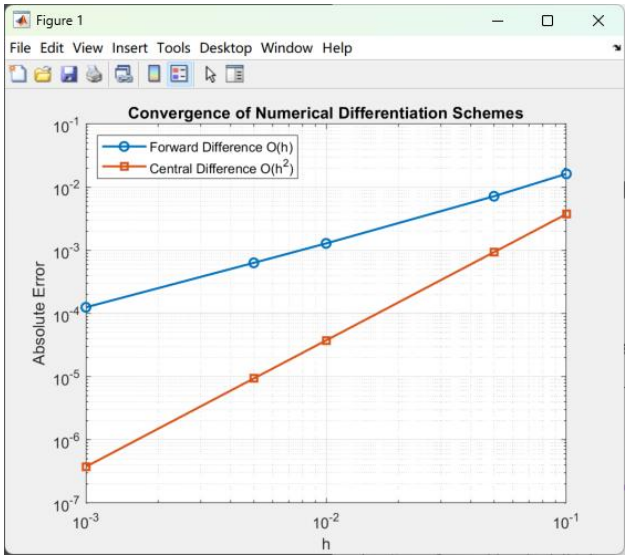
frequency region due to insufficient range, thus preventing aliasing error in the low-frequency coefficients.

Analysis of the Error Plot:

The plot illustrates the impact of the aliasing effect on the coefficients. It can be observed that the error is largest in the central low-frequency region. This can be viewed as a result of high frequencies folding back toward the center, causing the maximum impact on low frequencies. At approximately $j = \pm 23$, the error drops sharply; it is conjectured that this is near the Nyquist boundary, where certain high-frequency components fold back in a manner that results in the minimum magnitude of impact in this specific region.

3.

(a) . Following are the error of Forward Difference and Central Difference:



h	Forward Error	Central Error
0.1000	1.6258962253e-02	3.7485421130e-03
0.0500	7.1887110530e-03	9.3740886114e-04
0.0100	1.2875102709e-03	3.7499854159e-05
0.0050	6.3437629296e-04	9.3749908876e-06
0.0010	1.2537501046e-04	3.7500000172e-07

This section provides an analysis of the forward difference and central difference schemes. From a mathematical perspective, the error term for the forward difference, $R^2 f'(x_j)$, is directly proportional to the step size h (i.e., first-order accuracy, O(h)). This means that when h is reduced to 1/10, the error of the forward difference also decreases by approximately 1/10. In contrast, due to its symmetry, the central difference has an error term $R^3 f'(x_j)$ proportional to the square of the step size h (i.e., second-order accuracy, O(h^2)). Thus, when h is reduced to 1/10, the error of the central difference decreases by 1/100, theoretically providing much more accurate results.

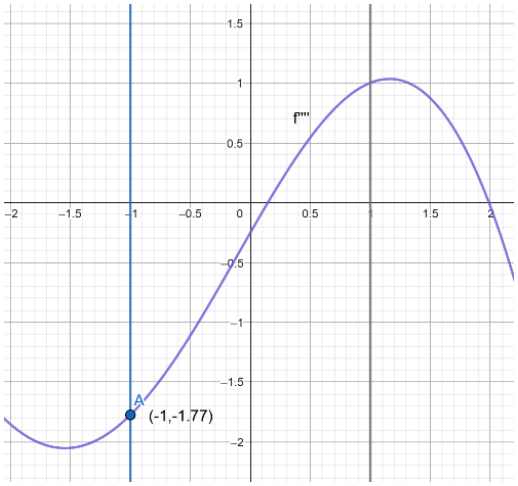
The numerical output from the program fits perfectly with these theoretical derivations. By conducting an error analysis at x = 0, it is evident that as h gradually decreases, the error of the forward difference maintains a roughly linear relationship with h. Meanwhile, the error of the central difference exhibits a rapid, quadratic decay. As h becomes smaller, the difference between the two schemes becomes more pronounced, and the calculations provided by the central difference become significantly more accurate.

(b) . Following are the error of Gaussian quadrature method and 1/3 Simpson's rules:

```
Exact Integral: -2.5876005968
Gauss 2-point:  -2.5856740220 (Error: 1.93e-03)
Simpson 1/3:    -2.5905134391 (Error: 2.91e-03)
```

For this problem, since N=1, we utilize a 2-point Gaussian quadrature scheme. The sampling points correspond to the two zeros of the 2nd-order Legendre polynomial, $x = \pm\frac{1}{\sqrt{3}}$。. Theoretically, a Gaussian quadrature with N+1 points can achieve an algebraic precision of degree 2N+1. Therefore, this method can provide exact analytical results for polynomials of degree 3 or lower. Based on the numerical output from our program, the error using the 2-point Gauss integration is only 1.93×10^{-3}. However, according to the professor's lecture notes, if this method is used to integrate polynomials higher than the third degree, the error can reach 14.8% or more, resulting in a sudden drop in accuracy. In such cases, 2-point Gauss integration may no longer be the optimal choice.

On the other hand, Simpson's 1/3 rule is a formula based on evenly spaced sampling points. It performs a quadratic interpolation using three equidistant points (x = -1, 0, 1). Based on the derived formula, the global error is approximately: $Error \approx -\frac{(x_N - x_1)}{180}\overline{f^{(4)}}h^4$. This indicates that the error of Simpson's 1/3 rule is proportional to the fourth power of the step size h. Thus, if h is reduced by 1/2, the theoretical error should decrease to 1/16 of its original value. Additionally, we observe that the error is directly proportional to the fourth derivative $f^{(4)}$. If the function fluctuates significantly within the integration domain—meaning the magnitude of the fourth derivative is large—the error of this method will increase accordingly.



The figure above illustrates the fourth derivative $f^{(4)}$ of $f(x)$. It can be seen that within the integration range, the maximum value of $f^{(4)}$ is only 1.77. This magnitude is not large enough to cause a substantial impact on the numerical error. Our program's output confirms this, as the error for Simpson's 1/3 rule is merely2.91×10^{-3}. Although this error is slightly higher than that of the Gaussian quadrature, it remains well within a reasonable and acceptable range for numerical simulation.

(c) . Following are the output of Bisection method and Newton-Raphson method：

```
--- Searching for Root 1 ---
Bisection: Root = -3.6688767026, Iterations = 33
Newton:    Root = -3.6688767027, Iterations = 3

--- Searching for Root 2 ---
Bisection: Root = -5.7591312526, Iterations = 33
Newton:    Root = -5.7591312526, Iterations = 4
```

In this section, I utilize the Bisection Method (Linear Convergence, α = 1) and the Newton-Raphson Method (Quadratic Convergence, α = 2) to perform numerical root approximation.

We initially define a permissible error tolerance (10^{-10}) and determine the starting intervals or initial positions for both methods. This process is crucial as it involves narrowing down the root's range beforehand to prevent converging to an incorrect root. Since both methods can approach the true value through near-infinite iterations, our comparison method involves setting a fixed error tolerance and observing how many iterations each method requires to enter this range. Fewer iterations indicate h igher efficiency and better accuracy under a fixed number of iterations.

Based on the output results, it is observed that since the Bisection method follows linear convergence ($\alpha = 1$) , it requires more than thirty iterations to reach a precision of 10^{-10}, demonstrating relatively low efficiency. In contrast, the Newton-Raphson method exhibits quadratic convergence ($\alpha = 2$) , where the error of the subsequent stage is proportional to the square of the current error. Consequently, it requires fewer than five iterations to enter the tolerance range.

Subsequently, the program was modified to a fixed number of iterations (iter = 15) to observe the difference in error between the two methods under identical computational effort.

```
Method               | Estimated Root       | Absolute Error
---------------------|----------------------|---------------------
Bisection (O(h))     | -3.668861389160156   | 1.53135e-05
Newton (O(h^2))      | -3.668876702710145   | 4.44089e-16
```

From the program output, it can be found that when both methods operate for 15 iterations simultaneously, the error of the Newton method (4.44×10^{-16}) is significantly smaller than that of the Bisection method (1.53×10^{-5}). This demonstrates that quadratic convergence is far superior to linear convergence in terms of computational efficiency.

Appendices

（以下 AI prompt 後續皆有針對 AI 輸出之程式碼進行進一步修改，並非輸入以下 prompt 後 AI 一次便能達到要求的程式碼）

B12502027_Q1_a (assisted by AI)

AI prompt:

寫一個 MATLAB 的程式分析 f(x) = cos(10x^2) e^(-x) 在區間 -1 ~ 1 的多項式插值誤差，並針對 N = 5, 10, 50, 100 等距節點使用 polyfit 擬合。每個 N 要呈現兩個圖，一個是疊加原函數跟新的擬合曲線，另一個呈現絕對誤差分佈。

Program:

```
% Q3 Interpolation Analysis: Fitting Overlay and Error Distribution
clear; clc;

f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000);
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);

    % 1. 生成等距插值點與計算插值多項式
    x_nodes = linspace(-1, 1, nodes);
```

```
    y_nodes = f(x_nodes);
    p = polyfit(x_nodes, y_nodes, nodes-1);
    y_interp = polyval(p, x_grid);
    error = abs(y_true - y_interp);

    % 建立新視窗或子圖 (左邊畫疊圖，右邊畫誤差)
    figure('Name', ['N = ', num2str(nodes)], 'NumberTitle', 'off');

    % --- 左圖：原函數 vs 擬合函數 ---
    subplot(1, 2, 1);
    plot(x_grid, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Original f(x)');
    hold on;
    plot(x_grid, y_interp, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Interpolated p(x)');
    plot(x_nodes, y_nodes, 'bo', 'MarkerFaceColor', 'b', 'DisplayName', 'Nodes');
    grid on;
    xlabel('x'); ylabel('f(x)');
    title(['Function Overlay (N=', num2str(nodes), ')']);
    legend('Location', 'best');
    ylim([-2, 3]); % 限制範圍以觀察主要特徵

    % --- 右圖：誤差分佈 (對數座標) ---
    subplot(1, 2, 2);
    semilogy(x_grid, error, 'm-', 'LineWidth', 1.5);
    grid on;
    xlabel('x'); ylabel('Absolute Error');
    title(['Error Distribution (N=', num2str(nodes), ')']);
    ylim([1e-16, 1e2]); % 觀察誤差等級
end
```

B12502027_Q1_b (assisted by AI)

AI prompt:

把上面那題的取點改成 Chebyshev Nodes

Program:

```
% Q3 Interpolation Analysis: Chebyshev Nodes Overlay and Error Distribution
clear; clc;

f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000);
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);

    % 1. 生成切比雪夫插值點 (Chebyshev Nodes)
    k = 1:nodes;
    x_cheb = cos((2*k-1) / (2*nodes) * pi);
    y_cheb = f(x_cheb);

    % 2. 進行多項式插值
    p = polyfit(x_cheb, y_cheb, nodes-1);
    y_interp = polyval(p, x_grid);
    error = abs(y_true - y_interp);
```

```
    % 建立新視窗
    figure('Name', ['N = ', num2str(nodes), ' (Chebyshev)'], 'NumberTitle', 'off');

    % --- 左圖：原函數 vs 切比雪夫擬合函數 ---
    subplot(1, 2, 1);
    plot(x_grid, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Original f(x)');
    hold on;
    plot(x_grid, y_interp, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Chebyshev p(x)');
    plot(x_cheb, y_cheb, 'go', 'MarkerFaceColor', 'g', 'DisplayName', 'Cheb Nodes');
% 綠色點代表切比雪夫節點
    grid on;
    xlabel('x'); ylabel('f(x)');
    title(['Function Overlay (N=', num2str(nodes), ')']);
    legend('Location', 'best');
    ylim([-2, 3]);

    % --- 右圖：誤差分佈 (對數座標) ---
    subplot(1, 2, 2);
    semilogy(x_grid, error, 'b-', 'LineWidth', 1.5);
    grid on;
    xlabel('x'); ylabel('Absolute Error');
    title(['Error Distribution (N=', num2str(nodes), ')']);
    ylim([1e-16, 1e2]);
end
```

B12502027_Q1_c (assisted by AI)

AI prompt:

把等距法的擬合方式改成多段的三次多項式擬合法

Program:

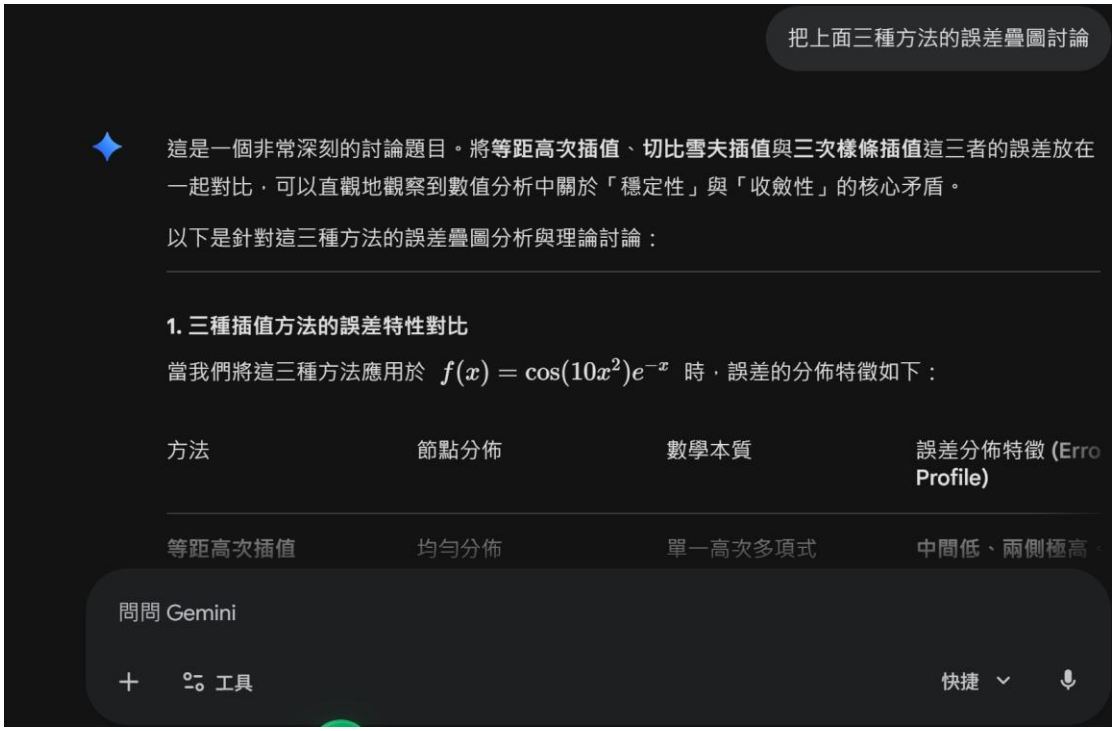
```
% Q3 Interpolation Analysis: Cubic Spline Overlay and Error Distribution
clear; clc;

f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000);
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);

    % 1. 生成等距插值點
    x_nodes = linspace(-1, 1, nodes);
    y_nodes = f(x_nodes);

    % 2. 進行 Cubic Spline 插值
    y_spline = spline(x_nodes, y_nodes, x_grid);
    error = abs(y_true - y_spline);

    % 建立新視窗
    figure('Name', ['N = ', num2str(nodes), ' (Cubic Spline)'], 'NumberTitle', 'off');
```

```
    % --- 左圖：原函數 vs 三次分段多項式擬合 ---
    subplot(1, 2, 1);
    plot(x_grid, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Original f(x)');
    hold on;
    plot(x_grid, y_spline, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Cubic Spline');
    plot(x_nodes, y_nodes, 'ko', 'MarkerFaceColor', 'y', 'DisplayName', 'Spline
Nodes'); % 黃色圓點代表節點
    grid on;
    xlabel('x'); ylabel('f(x)');
    title(['Function Overlay (N=', num2str(nodes), ')']);
    legend('Location', 'best');
    ylim([-2, 3]);

    % --- 右圖：誤差分佈 (對數座標) ---
    subplot(1, 2, 2);
    semilogy(x_grid, error, 'g-', 'LineWidth', 1.5);
    grid on;
    xlabel('x'); ylabel('Absolute Error');
    title(['Error Distribution (N=', num2str(nodes), ')']);
    ylim([1e-16, 1e2]);
end
```

B12502027_Q1_error discussion (assisted by AI)

AI prompt:

把上面三種方法的誤差疊圖討論



Program:

```
% Q3 Comparison: Equidistant vs Chebyshev vs Cubic Spline Error Distribution
clear; clc; close all;

% 定義目標函數
f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000); % 高密度網格用於計算誤差
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);

    % --- 1. 等距採點高次插值 (Equidistant High-degree) ---
    x_eq = linspace(-1, 1, nodes);
    y_eq = f(x_eq);
    p_eq = polyfit(x_eq, y_eq, nodes-1);
    y_interp_eq = polyval(p_eq, x_grid);
    err_eq = abs(y_true - y_interp_eq);

    % --- 2. 切比雪夫節點插值 (Chebyshev Nodes) ---
```

```
    k = 1:nodes;
    x_cheb = cos((2*k-1) / (2*nodes) * pi);
    y_cheb = f(x_cheb);
    p_cheb = polyfit(x_cheb, y_cheb, nodes-1);
    y_interp_cheb = polyval(p_cheb, x_grid);
    err_cheb = abs(y_true - y_interp_cheb);

    % --- 3. 三次樣條插值 (Cubic Spline) ---
    % 注意：Spline 通常使用等距點即可展現其穩定性
    y_interp_spline = spline(x_eq, y_eq, x_grid);
    err_spline = abs(y_true - y_interp_spline);

    % --- 繪圖討論 ---
    figure('Name', ['Error Comparison N = ', num2str(nodes)], 'Color', 'w');

    % 使用 semilogy 觀察誤差量級的巨大差異
    semilogy(x_grid, err_eq, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Equidistant Poly (Runge Risk)');
    hold on;
    semilogy(x_grid, err_cheb, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Chebyshev Poly (Minimax)');
    semilogy(x_grid, err_spline, 'g-', 'LineWidth', 1.2, 'DisplayName', 'Cubic Spline (Piecewise)');

    grid on;
    xlabel('x');
    ylabel('Absolute Error (log scale)');
    title(['Error Distribution Comparison (N = ', num2str(nodes), ')']);
    legend('Location', 'southoutside', 'Orientation', 'horizontal');

    % 固定 Y 軸範圍以便在不同 N 之間做對照
    ylim([1e-16, 1e2]);
    xlim([-1, 1]);
end
```

B12502027_Q2_b (assisted by AI)

AI prompt:

寫一個 MATLAB 程式碼針對高斯函數 g(x) = e^(-ax^2) 寫 DFT 程式，a=5 並在區間 [-pi, pi), N=64。要輸出圖

Program:

```
a = 5;
N = 64;
dx = 2*pi / N;
x = -pi : dx : pi-dx;

g_x = exp(-a * x.^2);

G_fft = fft(g_x);

g_tilde = fftshift(G_fft) / N;

j = -N/2 : N/2-1;

stem(j, abs(g_tilde), 'filled');
title(['DFT Coefficients g_j for a = ', num2str(a), ', N = ', num2str(N)]);
xlabel('Index j');
```

ylabel('g_j');
grid on;
hold on;

B12502027_Q2_d (assisted by AI)

AI prompt:

先用 FFT 做一次，再用 3/2 法則做一次，幫我輸出兩者結果的輸出圖

Program:

```
a = 2;
N = 64;
M = 3/2 * N;

%直接 FFT
dx1 = 2*pi/N;
x1 = -pi : dx1 : pi-dx1;
h_x1 = (exp(-a * x1.^2)).^2;
h_tilde_1 = fftshift(fft(h_x1)) / N;

%3/2 法則 / Zero-padding
g_x = exp(-a * x1.^2);
g_tilde = fftshift(fft(g_x)) / N;
```

```
g_tilde_padded = zeros(1, M);
idx = (M-N)/2 + 1 : (M+N)/2;
g_tilde_padded(idx) = g_tilde * (M/N);

g_fine = ifft(ifftshift(g_tilde_padded)) * M;
h_fine = g_fine.^2;

h_tilde_full = fftshift(fft(h_fine)) / M;
h_tilde_2 = h_tilde_full(idx);

% 比較誤差
aliasing_error_nonlinear = abs(h_tilde_1 - h_tilde_2);

figure;
semilogy(-N/2:N/2-1, aliasing_error_nonlinear, 'r', 'LineWidth', 1.5);
title('Isolated Aliasing Error due to Nonlinear Product');
xlabel('j'); ylabel('Absolute Difference');
grid on;
```

B12502027_Q3_a (assisted by AI)

AI prompt:

寫一個 MATLAB 程式比較 f(x) = 2sin(x) - 0.25e^x - 1 在 x=0 的前向差分跟中心差分的誤差。h 從 0.1 遞減至 0.001，輸出誤差對照表。

Program:

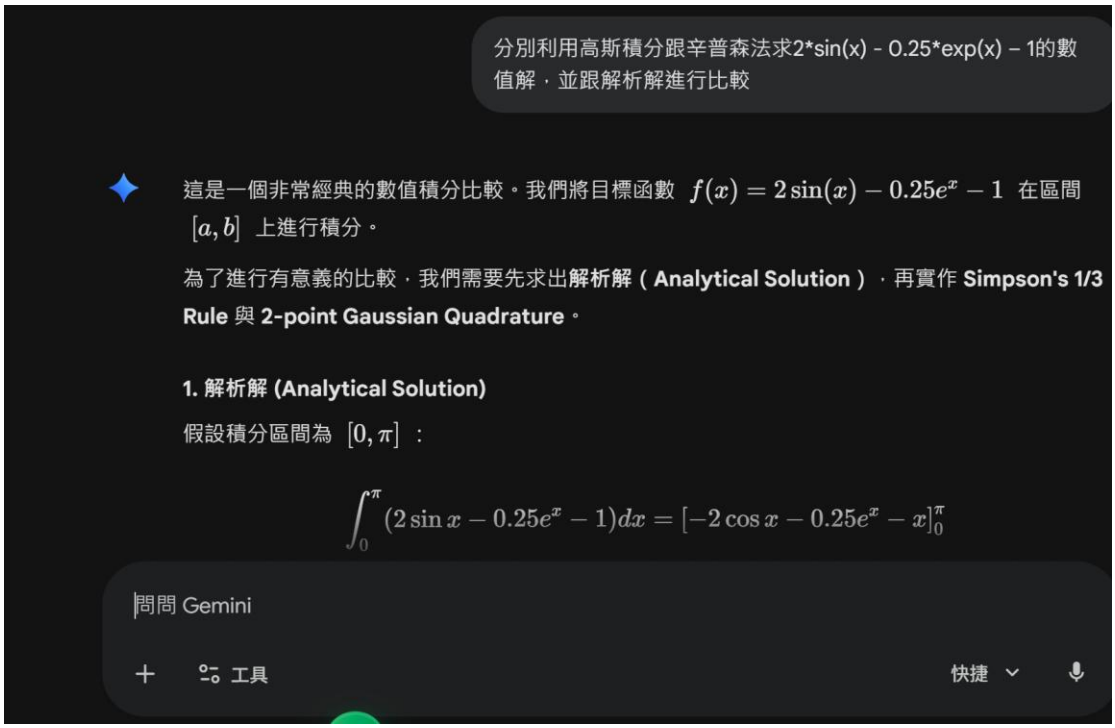
```
f = @(x) 2*sin(x) - 0.25*exp(x) - 1;
df_exact = @(x) 2*cos(x) - 0.25*exp(x);

x0 = 0;
h_values = [0.1, 0.05, 0.01, 0.005, 0.001];

error_forward = zeros(size(h_values));
error_central = zeros(size(h_values));

fprintf('%-10s | %-20s | %-20s\n', 'h', 'Forward Error', 'Central Error');

for i = 1:length(h_values)
    h = h_values(i);

    %前向差分
    df_forward = (f(x0 + h) - f(x0)) / h;
    error_forward(i) = abs(df_exact(x0) - df_forward);
```

```
    %中心差分
    df_central = (f(x0 + h) - f(x0 - h)) / (2 * h);
    error_central(i) = abs(df_exact(x0) - df_central);

    fprintf('%-10.4f | %-20.10e | %-20.10e\n', h, error_forward(i), error_central(i));
end

figure;
loglog(h_values, error_forward, '-o', 'LineWidth', 1.5, 'DisplayName', 'Forward
Difference O(h)');
hold on;
loglog(h_values, error_central, '-s', 'LineWidth', 1.5, 'DisplayName', 'Central
Difference O(h^2)');
grid on;
xlabel('h');
ylabel('Absolute Error');
title('Convergence of Numerical Differentiation Schemes');
legend('Location', 'northwest');
```

B12502027_Q3_b (assisted by AI)

AI prompt:

分別利用高斯積分跟辛普森法求 2*sin(x) - 0.25*exp(x) – 1 的數值解，並跟解析解進行比較



Program:

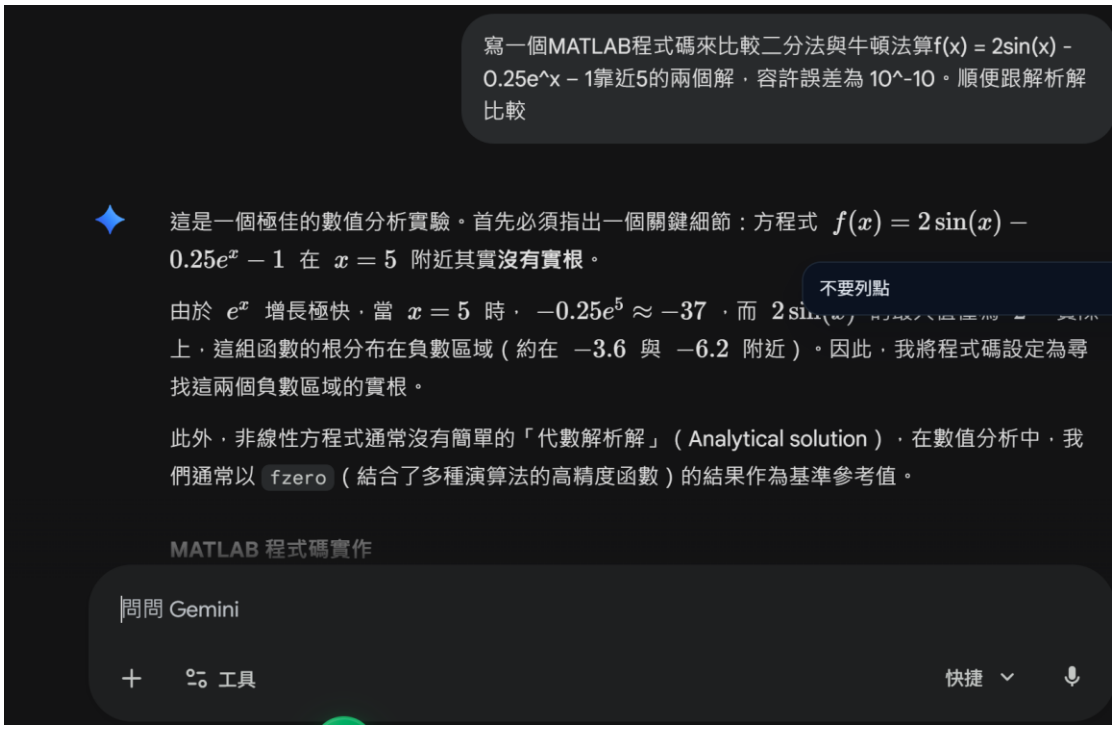
```
f = @(x) 2*sin(x) - 0.25*exp(x) - 1;

%解析解
exact_ans = (-2*cos(1) - 0.25*exp(1) - 1) - (-2*cos(-1) - 0.25*exp(-1) + 1);

% 高斯積分
x_gauss = [-1/sqrt(3), 1/sqrt(3)];
ans_gauss = f(x_gauss(1)) + f(x_gauss(2));

%辛普森法
h = 1;
ans_simpson = (h/3) * (f(-1) + 4*f(0) + f(1));

fprintf('Exact Integral: %.10f\n', exact_int);
fprintf('Gauss 2-point:    %.10f (Error: %.2e)\n', ans_gauss, abs(exact_int -
ans_gauss));
```

fprintf('Simpson 1/3: %.10f (Error: %.2e)\n', ans_simpson, abs(exact_int - ans_simpson));

B12502027_Q3_c (assisted by AI)

AI prompt:

寫一個 MATLAB 程式碼來比較二分法與牛頓法算 f(x) = 2sin(x) - 0.25e^x – 1 靠近 5 的兩個解，容許誤差為 10^-10。順便跟解析解比較



Program:

```
f = @(x) 2*sin(x) - 0.25*exp(x) - 1;
df = @(x) 2*cos(x) - 0.25*exp(x);

tol = 1e-10;
max_iter = 100;

search_data = {
    struct('name', 'Root 1', 'a', -4.5, 'b', -3.0, 'x0', -3.5),
    struct('name', 'Root 2', 'a', -6.5, 'b', -5.0, 'x0', -6.0)
};

for i = 1:length(search_data)
    data = search_data{i};
    fprintf('--- Searching for %s ---\n', data.name);
```

```
    %二分法
    a = data.a;
    b = data.b;
    iter_bisect = 0;
    while (b - a)/2 > tol && iter_bisect < max_iter
        c = (a + b) / 2;
        if f(a) * f(c) < 0
            b = c;
        else
            a = c;
        end
        iter_bisect = iter_bisect + 1;
    end
    root_bisect = (a + b) / 2;

    %牛頓法
    x_n = data.x0;
    iter_newton = 0;
    while abs(f(x_n)) > tol && iter_newton < max_iter
        x_n = x_n - f(x_n) / df(x_n);
        iter_newton = iter_newton + 1;
    end
    root_newton = x_n;

    % 輸出結果對比
    fprintf('Bisection: Root = %.10f, Iterations = %d\n', root_bisect, iter_bisect);
    fprintf('Newton:        Root = %.10f, Iterations = %d\n', root_newton,
iter_newton);
    fprintf('\n');
end
```

中文版本報告：

Q1（a）

分別針對 numbers of interpolation points = 5, 10, 50, 100 進行誤差分布的輸出以及分析。由圖中可發現，四張圖分別在節點處時，由於差值多項式強制要求通過這些點，所以誤差會趨近於 0，在誤差分布圖中會產生幾個瞬間下降的點，但由於網格無法達到無限密集，所以誤差並不會完美下降至 0。而排除這些固定數值的節點處，可明顯發現當 numbers of interpolation points 上升時，整體誤差分布呈現「中間低兩側高」的趨勢，可推斷若增加 numbers of interpolation points，對於中間區域（$x \approx 0$）的誤差穩定是有幫助的，但由於邊緣區域（$x \approx \pm 1$）為了強行擬合那些等距節點，在節點之間會產生較大的誤差，進而導致在邊緣區域的預測較不準確。總而言之，可發現即便我們持續增加 numbers of interpolation points 期望能試圖提升精度，也無法保證做到全域平均的收斂。

而在深入研究過龍格現象之後，我推測這幾張圖顯示出的效果並非龍格現象，龍格現象是指若我們漸漸增加採點數，兩側的誤差會瞬間飆升至 10 的 2～3 次方，甚至更高，但觀察 N=100 的圖可發現，即便兩側的誤差雖比中間還高，呈現出龍格現象所說的中間低兩側很高的趨勢，但最高處仍然在10^{-4}以下，並沒有誤差飆升的問題，由此可知對於此方程式，雖然等距採點法會造成誤差的不平均分配，但並不會造成龍格現象。

Q1（b）

同樣針對 numbers of interpolation points = 5, 10, 50, 100 進行誤差分布的輸出以及分析，但相較於(a)的平均分布節點，切比雪夫節點為一半圓的等分點投影在直線上的分布點，可達到中間疏兩邊密的效果。

由誤差分布圖可發現，此類節點分布透過增加在邊緣區域的採樣密度，有效壓制了震盪，可極大的改善邊緣區域誤差增大的問題。總而言之，若使用切比雪夫節點，隨著 N 的增加，誤差分布會呈現全域收斂的狀態，不論是中央還是邊緣皆能均勻地變小，有效的壓制住了誤差分布不均勻的問題，這也證明了節點分佈的選擇對於高次多項式插值的穩定性至關重要。

除此之外，若原函式在邊緣處存在龍格現象，此種採點法也能非常有效的避免，邊緣密集的採點能大幅度的減緩龍格現象誤差的突起，比起等距採點會有更好的效果。

Q1（c）

不同於(a)小題的高次多項式差值（單一曲線），Cubic Spline Interpolation 在每兩個節點之間建立一個三次多項式，並確保節點處一次導數和二次導數的連續。由四張誤差分布圖可發現，當 N 持續增大時，雖然誤差分布仍有些許中間底兩側高的趨勢，但相較於(a)小題的輸出結果，Cubic Spline Interpolation 已經能有效地趨緩兩側區域誤差值過大的問題，盡可能將全域誤差分布接控制在一個範圍之內。而隨著 N 的增加，全域誤差可達到均勻的下降，這證明了若需要使用等距採點的方式進行分析的話，相對於單一高次多項式，使用分段低次多項式會有更高的穩定性。
相同於切比雪夫節點法，這種分段低次多項式也可以減緩龍格現象在兩側的巨大誤差。

Q1 不同方法的誤差討論：

最後我將三個種方法的誤差進行疊圖並交叉比對討論，很明顯能看出當 N 值較小時，三種方法的誤差接相差不大，量級也差不多落在10^1左右，但隨著 N 增大，達到 N=50 以上時，很明顯看得出在中間部分，兩個單一高次多項式的誤差較分段低次多項式低上許多，相差接近10^9的量級，然而我們也可以發現在誤差平均分布的部分切比雪夫節點和 Cubic Spline Interpolation 所展現出的效果較等距節點好非常多，在接近 N=100 時甚至等距節點在邊緣的誤差已經和 Cubic Spline Interpolation 整體的誤差沒有特別大的差距了。

總而言之，若 N 值不斷往上增加，兩個單一高次多項式的誤差在中間段有較好的表現，但如果今天是 N 值已經足夠大，讓三種方法的誤差皆能落在可控範圍的話，進而若想追求誤差的平均分布，切比雪夫節點和 Cubic Spline Interpolation 則能提供交好的結果。

Q2（b）

印證：
根據(a)小題得出的解析解公式，當將 a = 5, N = 64 帶入計算時，可算出在 j = 0

時$\hat{f_j}(5) = \frac{1}{2\pi}\sqrt{\frac{\pi}{5}}e^{-\frac{0^2}{4\times5}} \approx 0.126$ ，與輸出圖表在 j = 0 時的波峰高度大致相符。

後續分別輸出 a = 2, 5, 10 三張圖並進行比較：

可發現當 a 值增大時，g 持續變成相對尖銳，而 g_2 分布則趨於平緩，符合傅立葉變換時域越尖銳，頻域就會越平緩理論。

Q2（c）

Truncation Error (截斷誤差)

造成原因：

真實的$G(s;a)$ 是連續且無限延伸的，但在計算 Fourier series 時卻僅保留有限頻率$j \in (-\frac{N}{2}, \frac{N}{2}-1)$，高頻部分被截斷所產生的誤差，造成$\frac{1}{2\pi}|G(s;a)|$和$|\hat{f_j}(a)|$理論上雖應保持一致，而兩者之間所存在的差異即為 Truncation Error。

誤差隔離：

若固定 N 並確保數值足夠大，再由小到大漸漸增加 a 值，便可觀察到當 a 值極小時，函式收斂速度較慢，若僅觀察中間區域而忽略高頻區域，則會發現$|\hat{f_j}(a)|$和理論值$\frac{1}{2\pi}|G(s;a)|$差異巨大。而當 a 值漸漸變大時，函式在高頻區域的收斂速度增加，$|\hat{f_j}(a)|$和$\frac{1}{2\pi}|G(s;a)|$之間的差異會明顯減小，這證明了 a 值控制著 Truncation Error。

Sampling / Aliasing Error (採樣/混疊誤差)

造成原因：

在計算$|\hat{f_j}(a)|$時仍假設連續的（積分形式），但在計算$|\hat{g}_j(a;N)|$時卻只針對在固定範圍內的 N 個離散點上取值，而根據 Nyquist Theorem，若某訊號超出頻譜範圍，高頻則會折返回來並偽裝成較低頻，並造成不同頻率的混雜，此時若比較$|\hat{f_j}(a)|$以及$|\hat{g}_j(a;N)|$較低頻時的數值，便可發現誤差的存在。

誤差隔離：

若固定一個較大的 a 值（例如：a = 10），並由小到大漸漸增加 N 值，會發現當 N 較小時，觀察範圍較小，且由於 a = 10 導致頻譜較寬，導致高頻折返回來疊加於低頻上，造成在低頻上$|\hat{g}_j(a;N)|$會明顯高於$|\hat{f_j}(a)|$。而當 N 逐漸增加時，超出範圍的數值漸漸縮小，誤差也快速下降。

Non-periodicity/Leakage Error (非週期性/洩漏誤差)

造成原因：

由於$G(s;a)$是連續且無邊界問題的，但在計算$|\hat{g}_j(a;N)|$時卻會受到邊界影響，若此時在邊界上拼接時產生了不連續，在週期化重疊時便會產生斷層。而根據傅立葉分析，任何斷層皆需要無限多個高頻波組成，此斷層會進而影響本應該集中在某個頻率的能量，讓其洩漏到整個頻譜上。

誤差隔離：

若同樣固定 N 值並由小到大漸漸增加 a 值，當 a 值較小時，高斯函數較為平緩，可能會在邊界處有非 0 的不連續產生，這時可觀察到有整體誤差抬升的趨勢。但若增加 a 值，讓高斯變得更尖銳，確保其在抵達邊界之前便趨近於 0，即可大幅降低此誤差。

Q2（d）

這題是先直接進行 FFT 計算 DFT 係數，得到$|\tilde{h}_{j,1}(a;N)|$，接著再將原始訊號進行 3/2 去混疊法則 後重新計算一次 DFT 係數$|\tilde{h}_{j,2}(a;N)|$，並比較兩者的差值便可隔離出非線性混疊誤差。

直接計算 ($|\tilde{h}_{j,1}(a;N)|$)：

假設$g(x;a)$的最高頻率成分是k_{max}，則其平方$h(x;a)=g^2(x;a)$的最高頻率成分$k'_{max}=2k_{max}$。若此時仍維持原本的 N 個採樣點，根據 (c) 小題提到的 Aliasing Error 裡的 Nyquist Theorem，這些超出範圍的高頻會折返回低頻區，造成嚴重的混疊誤差。

3/2 去混疊法則後再進行計算 ($|\tilde{h}_{j,2}(a;N)|$)：

其主要邏輯為首先針對長度 N 進行 FFT，轉換完後將 N 個頻譜係數放入一個更大的陣列 M = 3/2N 中，並在高頻區填入 0。此時將這些更多的 M 點做 IFFT 便可在時域得到更多採樣點的網格。若在此處理過的網格上進行計算，由於範圍較大讓多餘的高頻成分不會發生折返，接著再針對長度 M 再進行一次 FFT，完成後將多餘的部分截掉，剩下原始長度 N 所對應的係數。若將數據係數針對以上方法處理過後，高頻成分便不會因為範圍不夠而折返回低頻區，不會對低頻區的係數產生混疊誤差。

誤差圖表分析：

此圖表顯示了受到混疊效應對於係數的影響誤差。觀察圖表可發現，在中間低頻區域時誤差最大，可視為由於高頻皆往中間折返，讓低頻受到的混疊誤差影

響最大，而在j = ±23左右時誤差突然瞬間下降，推測此為 Nyquist 邊界附近，有些超出範圍的高頻成分在這段區域折返，讓此區域為影響幅度最小的地方。

Q3（a）

這邊針對 forward difference 和 central difference 進行分析。從數學計算上可得知，forward difference 的誤差項與步長 h 的一次方成正比，也就是說當 h 縮短為 1/10 時，forward difference 的誤差也會縮小 1/10。而 central difference 則因為其對稱性，誤差項與步長 h 的平方成正比，當 h 縮短為 1/10 時 central difference 的誤差會縮小 1/100，照理來說可以提供更精確的結果。

而程式的輸出也符合理論的推論，這邊針對 x = 0 的位置進行誤差分析，可明顯地發現當 h 值漸漸減小時，forward difference 和 h 值大致上呈現正比關係，而 central difference 卻呈指數型快速下降，當 h 值越小時，兩者差異越明顯，central difference 所提供的計算越準確。

Q3（b）

由於此題 N=1，使用的高斯積分為兩點高斯積分，並且採樣點為 2nd-order Legendre polynomial 的兩個根，$x = \pm\frac{1}{\sqrt{3}}$。而若使用 N+1 個採樣點的高斯積分其精度可達 N+1 階，所以本題在處理三次以及三次以下的多項式時可以精確地進行分析，從程式的誤差輸出結果可發現，若使用兩點高斯積分計算誤差僅為 1.93×10^{-3}。然而根據教授講義裡提到的，若拿兩點高斯積分去計算高於三階的多項式，其誤差可達 14.8%或以上，精度會瞬間下降，此時高斯積分便不是一個較好的選擇。

而 1/3 Simpson's rules 則是一個等距採點的公式，分別針對三個等距點 $(x = -1,0,1)$ 進行二項式插值。根據推導的公式可知，$Error \approx -\frac{(x_N - x_1)}{180}\overline{f^{(4)}}h^4$，亦即 1/3 Simpson's rules 的誤差與步長 h 的四次方成正比，若步長 h 下降 1/2，誤差理論會下降至原本的 1/16。除此之外，我們也可以發現 1/3 Simpson's rules 的誤差與四階導數$f^{(4)}$成正比，若該函數在積分區域內波動巨大，也就是四階導數數值很大，此方法的誤差便也會提升。下圖為$f(x)$的四階導數$f^{(4)}$，可知在積分範圍內，$|f^{(4)}|$的數值最大也才 1.77，不至於對誤差產生太大的影響，由程式的誤差輸出結果也可發現，使用 1/3 Simpson's rules 的誤差僅為 2.91×10^{-3}，雖然較高斯積分大，但也屬於合理範圍。

Q3（c）

這邊分別使用 Bisection Method (Linear Convergence, $\alpha = 1$) 和 Newton-Raphson Method (Quadratic Convergence, $\alpha = 2$) 進行逼近求根。

一開始先定義好可容誤差範圍 (10^{-10}) 並訂定二分法和牛頓法的起始區間和位置，此過程非常重要，需要先大致上鎖定根的範圍以免不小心找錯根，由於兩者在接近無限多次迭代後皆可逼近真實數值，所以我們在這邊比較的方法是訂定一個誤差容許範圍，然後分別觀察兩種方法分別進行多少次迭代之後能進入這個容許範圍，次數越少的代表其越有效率且在固定的迭代次數下也能較準確。

觀察輸出結果可發現，由於二分法為線性收斂 $(\alpha = 1)$，在到達10^{-10}精度的過程中會需要高達三十次的逼近，效率非常差。而牛頓法為二階收斂 $(\alpha = 2)$，下一階段的誤差與當前誤差的平方成正比，所以僅需不到五次迭代便可進入容許範圍。

後續將程式改成固定迭代次數 (iter = 15)，並觀察在相同迭代次數下，兩種方法在誤差上的差異。

由程式輸出結果可發現，當兩種方法同時皆操作 15 次迭代時，牛頓法的誤差4.44×10^{-16}遠小於二分法的誤差1.53×10^{-5}，由此可知二階收斂在效率上是遠高於線性收斂的。